

Word2vec

FCEFyN - UNC - Cátedra de Inteligencia Artificial

- Autor/es: Ing. Sandro Comerci
- Directora: Dra. Laura Cecilia Diaz Davila (LIDeSIA - FCEFyN)

En la última década, el Procesamiento de Lenguaje Natural (NLP) ha avanzado de manera vertiginosa en la forma en que representamos y modelamos el lenguaje. Un punto de partida clave fueron los **embeddings de palabras estáticos**, como **Word2Vec**, que permitieron capturar relaciones de significado a partir del contexto en el que las palabras aparecen.

Representación de Palabras y Word2Vec

Contexto y Representación Vectorial

En los primeros enfoques de NLP, las palabras se representaban con métodos muy básicos como **one-hot encoding** o **bag-of-words (BoW)**:

- **One-hot encoding:** cada palabra del vocabulario se convierte en un vector largo y disperso, con un único `1` en la posición correspondiente y `0` en todas las demás.
    - Problema: estos vectores tienen la misma dimensión que el vocabulario, son en su mayoría ceros y **no reflejan ninguna relación semántica** entre palabras.

```
1 vocab = ["queen", "woman", "girl", "boy", "man", "king"]
2 one_hot = {w: [1 if i == j else 0 for i in range(len(vocab))] for j, w i
```

```
   3 one_hot
```

```
{'queen': [1, 0, 0, 0, 0, 0, 0],
 'woman': [0, 1, 0, 0, 0, 0, 0],
 'girl': [0, 0, 1, 0, 0, 0, 0],
 'boy': [0, 0, 0, 1, 0, 0, 0],
 'king': [0, 0, 0, 0, 0, 0, 1],
 'man': [0, 0, 0, 0, 0, 1, 0]}
```

Bag-of-Words (BoW): en este enfoque, cada unidad de texto (puede ser una oración, un párrafo o un documento completo) se representa como un vector que indica cuántas veces aparece cada palabra del vocabulario.

- Problema: aunque es sencillo, **pierde completamente el orden y el contexto**. Trata las palabras como si fueran independientes, ignorando la estructura del lenguaje.

```
 1 # Corpus de ejemplo
 2 corpus = ["el perro ladra", "el gato maúlla"]
 3
 4 # Construimos el vocabulario (palabras unicas en todo el corpus)
 5 vocab = sorted(set(" ".join(corpus).split()))
 6 print("Vocabulario:", vocab)
 7
 8 # Creamos un diccionario {palabra: índice}
 9 word_to_index = {word: i for i, word in enumerate(vocab)}
10
11 # Representamos cada documento como vector de frecuencias
12 bow_vectors = []
13 for doc in corpus:
14     vector = [0] * len(vocab)
15     for word in doc.split():
16         vector[word_to_index[word]] += 1
17     bow_vectors.append(vector)
18
19 # Resultados
20 for doc, vec in zip(corpus, bow_vectors):
21     print(f"'{doc}' → {vec}")
```

```
Vocabulario: ['el', 'gato', 'ladra', 'maúlla', 'perro']
'el perro ladra' → [1, 0, 1, 0, 1]
'el gato maúlla' → [1, 1, 0, 1, 0]
```

Ambos enfoques marcaron los primeros pasos en la representación numérica del lenguaje, pero resultaban muy limitados.
Sus principales problemas eran:

- Producían vectores **enormes y dispersos** (la mayoría de los valores eran ceros).
- **Ignoraban el orden sintáctico**, tratando a las palabras como independientes.

- No capturaban **relaciones de significado**: por ejemplo, no había forma de reflejar que “excelente” y “estupendo” son términos cercanos en cuanto a sentido.

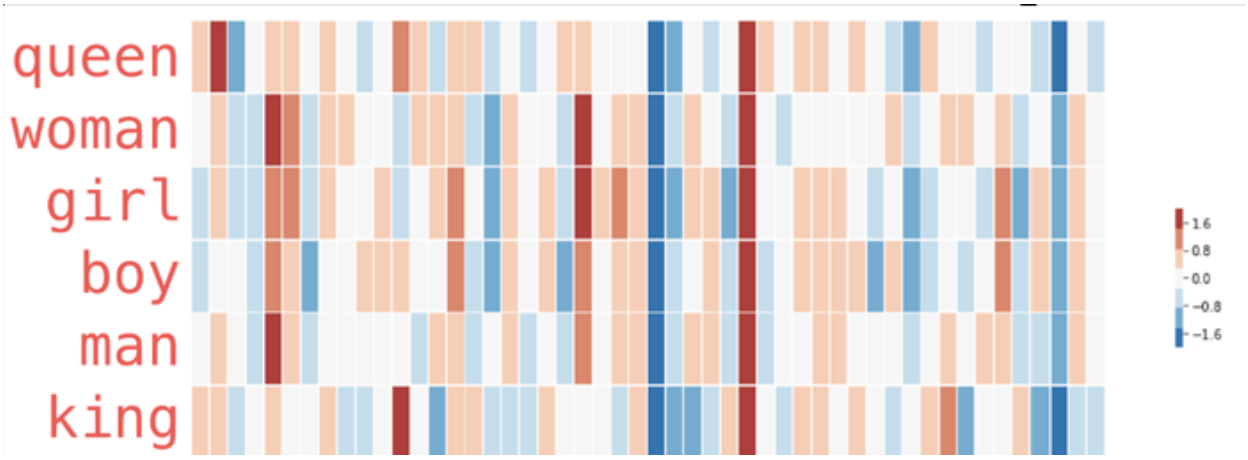
En otras palabras, estas técnicas podían contar palabras, pero no podían comprenderlas.

Para superar estas limitaciones surge la idea de los **embeddings densos**: representar cada palabra mediante un vector compacto de baja dimensión (típicamente entre 50 y 300 valores) en lugar de usar vectores enormes y dispersos.

En este espacio vectorial, las **distancias reflejan relaciones semánticas**:

- Palabras que aparecen en contextos similares terminan con vectores cercanos.
- Palabras no relacionadas aparecen más alejadas.

Este enfoque se conoce como **representación distribuida del significado**, y permite que el modelo capture matices semánticos y asociaciones que antes eran imposibles de representar con métodos simples como One-Hot o Bag-of-Words.



Antes de aprender tales embeddings, normalmente se realiza preprocesamiento del texto: tokenización (segmentar el texto en unidades básicas como palabras o subpalabras) y lematización o stemming (normalizar cada palabra a su forma base o raíz). Estos pasos reducen la variabilidad del texto y preparan los datos para entrenar los modelos de embeddings.

```
!pip install contractions
!pip install googletrans
!pip install gensim
```

Show hidden output

Lematización:

- La lematización es el proceso de reducir una palabra a su forma base o raíz, conocida como lema. Por ejemplo, las palabras "corriendo", "corre" y "correrá" tienen el mismo lema "correr".
- El objetivo de la lematización es normalizar las palabras para que las variantes de una misma palabra se consideren iguales en el análisis, lo que ayuda a mejorar la precisión de los modelos de NLP.

Stopwords:

- Las stopwords son palabras muy comunes en un idioma que a menudo no aportan un significado adicional al texto y se pueden eliminar sin afectar la comprensión del mismo. Ejemplos de stopwords en inglés son "the", "and", "is", etc.

Categoría gramatical	Ejemplos comunes	Función principal
Artículos	el, la, los, las, un, una	Determinan al sustantivo
Preposiciones	a, ante, bajo, con, contra, de	Relacionan palabras y sintagmas
Conjunciones	y, o, pero, aunque, porque	Unen palabras u oraciones
Pronombres	él, ella, lo, la, me, nos	Sustituyen al sustantivo
Adverbios comunes	no, sí, ya, aún, también	Modifican verbos, adjetivos u otros adverbios
Determinantes	este, ese, aquel, mi, tu	Acompañan al sustantivo especificando su referencia
Interjecciones simples	¡ah!, ¡eh!, ¡oh!	(Algunas pocas veces aparecen, pero suelen omitirse)

- La eliminación de stopwords es importante para reducir el ruido en los datos y mejorar la eficiencia de los modelos de NLP al centrarse en las palabras más significativas del texto.

```
import pandas as pd
import nltk
from nltk.stem import WordNetLemmatizer
from gensim.models.phrases import Phrases, Phraser
from gensim.models import Word2Vec
import re
import string
import contractions
import multiprocessing
from nltk.corpus import stopwords

nltk.download('wordnet')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

Limpieza del texto

- Normalización de datos
- Eliminación de ruido
- Reducción del vocabulario
- Mejora de la precisión

clean_text_round1: Limpia el texto convirtiéndolo a minúsculas, eliminando texto entre corchetes, signos de puntuación y palabras con números.

clean_text_round2: Elimina comillas, puntos suspensivos, y saltos de línea del texto.

```
def clean_text_round1(text):
    text = text.lower()
    text = re.sub('\[.*?¿\]\%', ' ', text)
    text = re.sub('[%s]' % re.escape(string.punctuation), ' ', text)
    text = re.sub('\w*\d\w*', '', text)
    return text

def clean_text_round2(text):
    text = re.sub('[‘’“”…«»]', '', text)
    text = re.sub('\n', ' ', text)
    return text
```

```
<>:3: SyntaxWarning: invalid escape sequence '\['
<>:5: SyntaxWarning: invalid escape sequence '\w'
<>:3: SyntaxWarning: invalid escape sequence '\['
<>:5: SyntaxWarning: invalid escape sequence '\w'
/tmp/ipython-input-3910757363.py:3: SyntaxWarning: invalid escape sequence '\
  text = re.sub('\[.*?¿\]\%', ' ', text)
/tmp/ipython-input-3910757363.py:5: SyntaxWarning: invalid escape sequence '\
  text = re.sub('\w*\d\w*', '', text)
```

```
lemmatizer = WordNetLemmatizer()
sw = stopwords.words('english')
```

```
df = pd.read_csv('/content/drive/MyDrive/Inteligencia Artificial/Materia
print(df.head())

# Asegurarse de que todas las entradas en 'spoken_words' son cadenas de
df['spoken_words'] = df['spoken_words'].astype(str)

# Aplicar las funciones de limpieza usando funciones lambda (rounds)
round0 = lambda x: contractions.fix(x)
round1 = lambda x: clean_text_round1(x)
round2 = lambda x: clean_text_round2(x)

# Aplicar cada ronda de limpieza secuencialmente
df['spoken_words'] = df['spoken_words'].apply(round0)
df['spoken_words'] = df['spoken_words'].apply(round1)
df['spoken_words'] = df['spoken_words'].apply(round2)

```

```
17
18 df.head()
```

```
       raw_character_text                                     spoken_words
0             Miss Hoover  No, actually, it was a little of both. Sometim...
1            Lisa Simpson                             Where's Mr. Bergstrom?
2             Miss Hoover  I don't know. Although I'd sure like to talk t...
3            Lisa Simpson                          That life is worth living.
4  Edna Krabappel-Flanders  The polls will be open from now until the end ...
```

	raw_character_text	spoken_words
0	Miss Hoover	no actually it was a little of both sometim...
1	Lisa Simpson	where is mr bergstrom
2	Miss Hoover	i do not know although i would sure like to t...
3	Lisa Simpson	that life is worth living
4	Edna Krabappel-Flanders	the polls will be open from now until the end ...

Tokenización de las frases

- Convierte cada frase en una lista de palabras, tokenizando el texto por espacios. input_data se convierte en una lista de listas de palabras.

```
1 input_data = [row.split() for row in df['spoken_words']]
2 input_data[0]
```

```
['no',
 'actually',
 'it',
 'was',
 'a',
 'little',
 'of',
 'both',
 'sometimes',
 'when',
 'a',
 'disease',
 'is',
 'in',
 'all',
 'the',
 'magazines',
 'and',
 'all',
 'the',
 'news',
 'shows',
 'it',
 'is',
 'only',
 'natural',
 'that',
```

```
 'you',
 'think',
 'you',
 'have',
 'it']
```

Construcción de bigramas

"El gato está sobre la mesa"

Los bigramas de esta oración serían:

- "El gato"
- "gato está"
- "está sobre"
- "sobre la"
- "la mesa"

Cada bigrama representa dos palabras consecutivas en la oración original.

Captura de Contexto Local

```
phrases = Phrases(input_data, min_count=30)
bigram = Phraser(phrases)
sentences = bigram[input_data]
sentences[0]
```

```
['no',
 'actually',
 'it',
 'was',
 'a',
 'little',
 'of',
 'both',
 'sometimes',
 'when',
 'a',
 'disease',
 'is',
 'in',
 'all',
 'the',
 'magazines',
 'and',
 'all',
 'the',
 'news',
 'shows',
 'it',
 'is',
 'only',
 'natural',
 'that',
 'you',
```

```
 'think',
 'you',
 'have',
 'it']
```

```
for sentence in sentences:
    for word in sentence:
        if "_" in word:
            print(f"Bigrama encontrado: {word}")
```

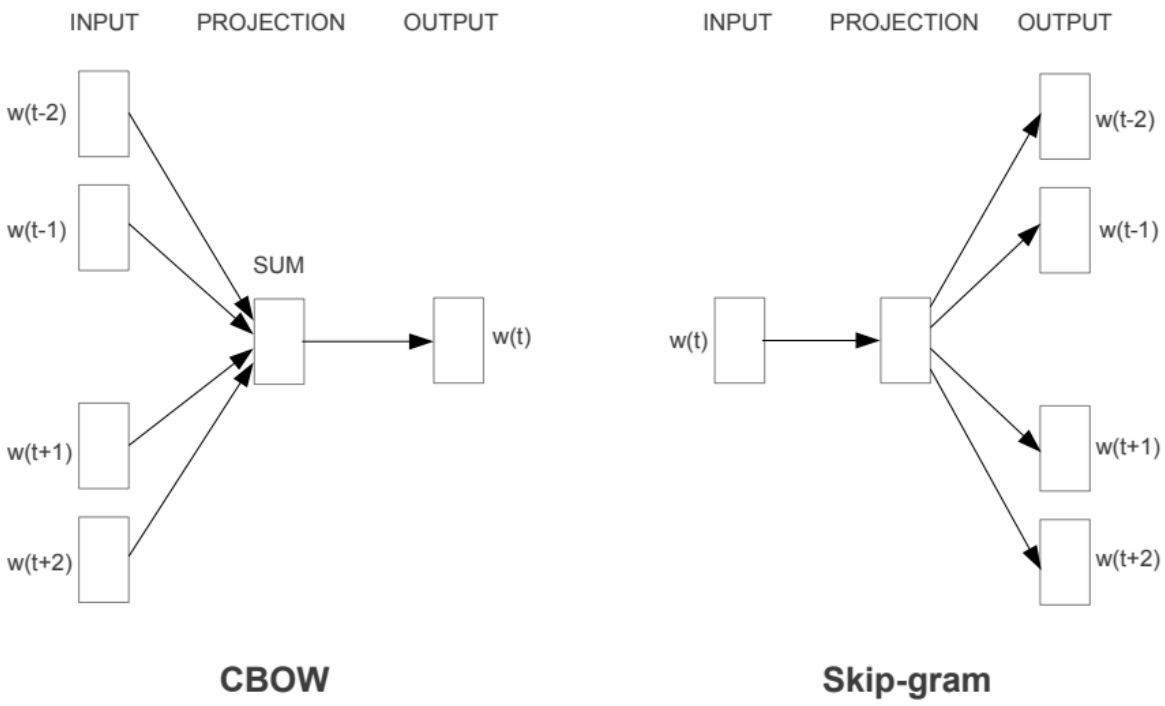
```
Streaming output truncated to the last 5000 lines.
Bigrama encontrado: cannot_believe
Bigrama encontrado: next_week
Bigrama encontrado: an_hour
Bigrama encontrado: have_been
Bigrama encontrado: kind_of
Bigrama encontrado: so_fast
Bigrama encontrado: do_not
Bigrama encontrado: excuse_me
Bigrama encontrado: have_been
Bigrama encontrado: do_not
Bigrama encontrado: do_not
Bigrama encontrado: let_us
Bigrama encontrado: your_father
Bigrama encontrado: do_not
Bigrama encontrado: have_been
Bigrama encontrado: am_glad
Bigrama encontrado: even_though
Bigrama encontrado: talking_about
Bigrama encontrado: talking_about
Bigrama encontrado: do_not
Bigrama encontrado: how_bout
Bigrama encontrado: do_not
Bigrama encontrado: each_other
Bigrama encontrado: do_not
Bigrama encontrado: am_sorry
Bigrama encontrado: do_not
Bigrama encontrado: woo_hoo
Bigrama encontrado: da_da
Bigrama encontrado: da_da
Bigrama encontrado: da_da
Bigrama encontrado: take_care
Bigrama encontrado: worse_than
Bigrama encontrado: at_least
Bigrama encontrado: do_not
Bigrama encontrado: lisa_simpson
Bigrama encontrado: y_ello
Bigrama encontrado: do_not
Bigrama encontrado: supposed_to
Bigrama encontrado: someone_who
Bigrama encontrado: do_not
Bigrama encontrado: woo_hoo
Bigrama encontrado: america_s
Bigrama encontrado: world_war
Bigrama encontrado: dear_god
Bigrama encontrado: cannot_believe
Bigrama encontrado: ay_carumba
Bigrama encontrado: ay_carumba
Bigrama encontrado: do_not
```

```
Bigrama encontrado: best_friend
Bigrama encontrado: do_not
Bigrama encontrado: all_right
Bigrama encontrado: super_bowl
Bigrama encontrado: do_not
Bigrama encontrado: do_not
Bigrama encontrado: super_bowl
Bigrama encontrado: trying_to
Bigrama encontrado: kind_of
```

Word2Vec

En 2013, Tomas Mikolov y su equipo presentaron **Word2Vec**, un modelo neuronal sencillo (con una sola capa oculta) diseñado para aprender embeddings de palabras de manera **eficiente y de alta calidad**.

Word2Vec se entrena a partir del **contexto en el que aparecen las palabras**, y ofrece dos arquitecturas principales:



Construcción del vocabulario

- **CBOW (Continuous Bag-of-Words):**
  El modelo recibe como entrada las palabras de contexto (las que rodean a la palabra objetivo) y debe **predecir la palabra central**.

  > Ejemplo: en la frase *"el ___ ladra fuerte"*, con contexto `[el, ladra, fuerte]`, el modelo debe predecir *"perro"*. (sg=0)

- **Skip-gram:**
  El modelo recibe como entrada una palabra central y debe **predecir las palabras de su contexto cercano**.

  > Ejemplo: si la palabra central es *"perro"*, el modelo debe predecir que en su vecindad aparecen palabras como *"el"* y *"ladra"*.
  > (sg=1)

```
cores = multiprocessing.cpu_count()
w2v_model = Word2Vec(sg=0, min_count=20, window=2, vector_size=300, samp
```

```
w2v_model.build_vocab(sentences)
# Obtener las palabras del vocabulario
words = list(w2v_model.wv.key_to_index.keys())
print (len(words))
```

```
4318
```

```
w2v_model.train(sentences, total_examples=w2v_model.corpus_count, epochs
w2v_model.init_sims(replace=True)
```

```
/tmp/ipython-input-3234137765.py:2: DeprecationWarning: Call to deprecated `i
  w2v_model.init_sims(replace=True)
WARNING:gensim.models.keyedvectors:destructive init_sims(replace=True) deprec
```

Palabras más cercanas en el espacio de embeddings

```
w2v_model.wv.most_similar(positive=["homer"])
```

```
[('marge', 0.5010542869567871),
 ('dad', 0.4555928707122803),
 ('homie', 0.4531590938568115),
 ('mom', 0.4006149172782898),
 ('lisa', 0.3823198676109314),
 ('dear_lord', 0.3594253957271576),
 ('becky', 0.3578633666038513),
 ('honey', 0.357191264629364),
 ('reverend_lovejoy', 0.3556356430053711),
 ('mr_burns', 0.35553038120269775)]
```

```
w2v_model.wv.most_similar(negative=["homer"])
```

```
[('pole', 0.28931981325149536),
 ('known_as', 0.27104973793029785),
 ('played', 0.2437598705291748),
 ('uses', 0.23563413321971893),
 ('congress', 0.23024573922157288),
 ('leaves', 0.22917315363883972),
 ('hits', 0.2258554846048355),
 ('texas', 0.22228050231933594),
```

```
 ('hail', 0.22134463489055634),
 ('including', 0.216420516371727)]
```

```
w2v_model.wv.most_similar(positive=["homer", "marge", "bart", "lisa", "b
```

```
[('mom', 0.5834949612617493),
 ('dad', 0.5291776657104492),
 ('maggie', 0.5242363214492798),
 ('homie', 0.5223982334136963),
 ('honey', 0.49859869480133057),
 ('milhouse', 0.46158236265182495),
 ('becky', 0.4486558437347412),
 ('dr_hibbert', 0.4396235942840576),
 ('sweetie', 0.43065086007118225),
 ('jessica', 0.42579010128974915)]
```

```
import plotly.graph_objs as go
from sklearn.decomposition import PCA
import numpy as np

# Seleccionar palabra específica y sus vecinas (aumentamos el número de
target_word = 'homer'  # Reemplaza con la palabra de interés
neighbors = w2v_model.wv.most_similar(target_word, topn=50)  # Obtener l
words = [target_word] + [neighbor[0] for neighbor in neighbors]
word_vectors = np.array([w2v_model.wv[word] for word in words])

# Reducir dimensiones a 3D con PCA
pca = PCA(n_components=3)
word_vectors_pca = pca.fit_transform(word_vectors)

# Crear trazado 3D con Plotly
trace = go.Scatter3d(
    x=word_vectors_pca[:, 0],
    y=word_vectors_pca[:, 1],
    z=word_vectors_pca[:, 2],
    mode='markers+text',
    text=words,
    marker=dict(size=5, color='blue', opacity=0.8),  # Ajustar el tamaño
    textposition="top center"
)

# Destacar la palabra objetivo en rojo
trace_target = go.Scatter3d(
    x=[word_vectors_pca[0, 0]],
    y=[word_vectors_pca[0, 1]],
    z=[word_vectors_pca[0, 2]],
    mode='markers+text',
    text=[target_word],
    marker=dict(size=10, color='red'),
    textposition="top center"
)

# Crear el layout
layout = go.Layout(
    title="Visualización de un Espacio Vectorial Más Grande",
```

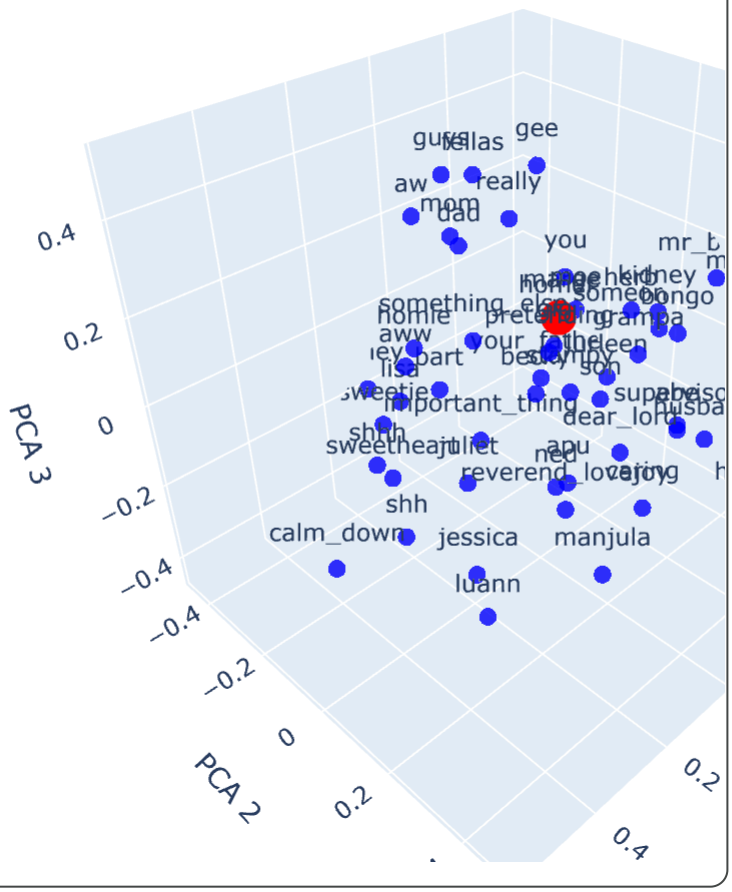
```
    scene=dict(
        xaxis_title="PCA 1",
        yaxis_title="PCA 2",
        zaxis_title="PCA 3"
    ),
    margin=dict(l=0, r=0, b=0, t=50),
)

# Combinar los trazos
fig = go.Figure(data=[trace, trace_target], layout=layout)

# Mostrar la gráfica interactiva
fig.show()

```

Visualización de un Espacio Vectorial Más Grande




```
import plotly.graph_objs as go
import numpy as np
from scipy.spatial.distance import cosine, cityblock

# Seleccionar las palabras de interés
words_of_interest = ['homer', 'marge', 'bart', 'lisa']  # Puedes agregar

# Obtener los vectores de las palabras
word_vectors = np.array([w2v_model.wv[word] for word in words_of_interes

```

```
# Crear trazado 3D con Plotly usando las dimensiones originales
trace_words = go.Scatter3d(
    x=word_vectors[:, 0],
    y=word_vectors[:, 1],
    z=word_vectors[:, 2],  # Usamos las primeras 3 dimensiones
    mode='markers+text',
    text=words_of_interest,
    marker=dict(size=10, color='blue'),
    textposition="top center"
)

# Lista para almacenar las líneas entre palabras
trace_lines = []

# Calcular y dibujar líneas para cada par de palabras
for i in range(len(words_of_interest)):
    for j in range(i+1, len(words_of_interest)):
        # Obtener los vectores de las palabras
        vector1 = word_vectors[i]
        vector2 = word_vectors[j]

        # Calcular distancias
        euclidean_distance = np.linalg.norm(vector1 - vector2)
        cosine_similarity = 1 - cosine(vector1, vector2)  # 1 - cosine p
        manhattan_distance = cityblock(vector1, vector2)

        # Mostrar las distancias en la consola
        print(f"Distancias entre '{words_of_interest[i]}' y '{words_of_i
        print(f"  Euclidiana: {euclidean_distance}")
        print(f"  Similitud de coseno: {cosine_similarity}")
        print(f"  Distancia Manhattan: {manhattan_distance}")
        print()

        # Dibujar línea entre las palabras para representar la distancia
        trace_line = go.Scatter3d(
            x=[vector1[0], vector2[0]],
            y=[vector1[1], vector2[1]],
            z=[vector1[2], vector2[2]],
            mode='lines',
            line=dict(color='gray', width=2),
            text=f"Euclidiana: {euclidean_distance:.2f}",
            hoverinfo="text"
        )
        trace_lines.append(trace_line)

# Crear el layout
layout = go.Layout(
    title="Comparación de Distancias en el Espacio Vectorial",
    scene=dict(
        xaxis_title="Vector Dimension 1",
        yaxis_title="Vector Dimension 2",
        zaxis_title="Vector Dimension 3"
    ),
    margin=dict(l=0, r=0, b=0, t=50),
)
```

```
66
67 # Combinar los trazos de puntos y líneas
68 fig = go.Figure(data=[trace_words] + trace_lines, layout=layout)
69
70 # Mostrar la gráfica interactiva
71 fig.show()
72
```

```
Distancias entre 'homer' y 'marge':
  Euclidiana: 0.9989451169967651
  Similitud de coseno: 0.5010542869567871
  Distancia Manhattan: 13.873126983642578

Distancias entre 'homer' y 'bart':
  Euclidiana: 1.140315055847168
  Similitud de coseno: 0.3498408004594089
  Distancia Manhattan: 15.516556739807129

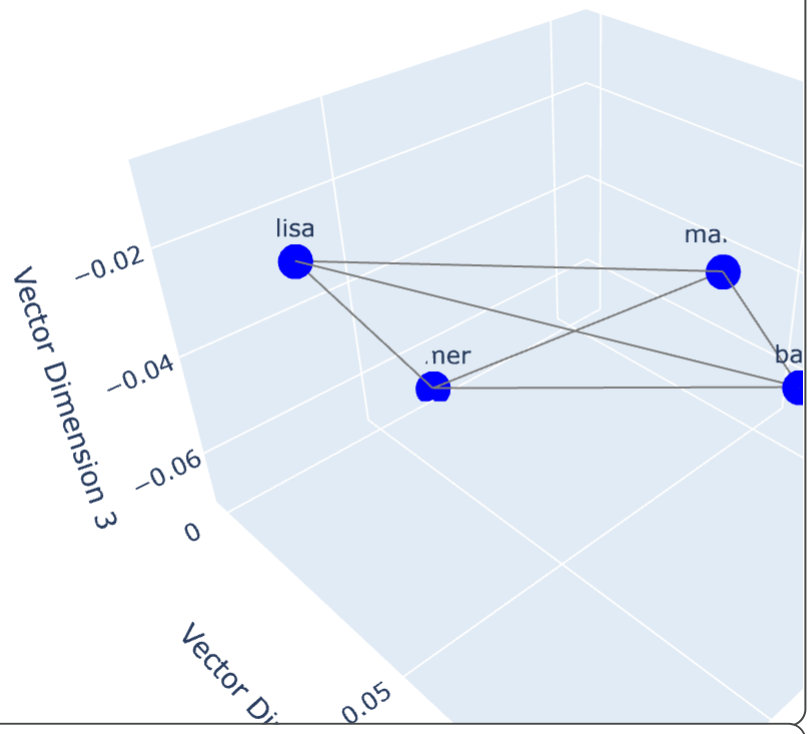
Distancias entre 'homer' y 'lisa':
  Euclidiana: 1.1114675998687744
  Similitud de coseno: 0.3823198974132538
  Distancia Manhattan: 15.10865592956543

Distancias entre 'marge' y 'bart':
  Euclidiana: 1.1861680746078491
  Similitud de coseno: 0.29650250960893976
  Distancia Manhattan: 16.36084747314453

Distancias entre 'marge' y 'lisa':
  Euclidiana: 1.1628950834274292
  Similitud de coseno: 0.32383739948272705
  Distancia Manhattan: 16.290218353271484

Distancias entre 'bart' y 'lisa':
  Euclidiana: 0.8641032576560974
  Similitud de coseno: 0.6266626902420134
  Distancia Manhattan: 11.924253463745117
```

Comparación de Distancias en el Espacio Vectorial



```
1 analogous_words = w2v_model.wv.mostpositive=['bart', '_similar(marge'],
2 print("La palabra que completa la analogía homer es a bart como marge es a ")
```

vec("bart")+vec("marge")−vec("homer") = vec("lisa")

```
1 print(w2v_model.wv.most_similar(positive=["burns", "smithers"], negative
```

```
[('waylon', 0.3694535493850708)]
```

```
1 print(w2v_model.wv.most_similar(positive=["homer", "moe"], negative=["ba
```

```
[('dad', 0.40101078152656555)]
```

```
1 print(w2v_model.wv.most_similar(positive=["maggie", "marge"], negative=[
```

```
[('homer', 0.34902775287628174)]
```

```
1 print(w2v_model.wv.most_similar(positive=["apu", "beer"], negative=["sto
```

```
[('baron', 0.3830994963645935)]
```

```
1 print(w2v_model.wv.most_similar(positive=["bart", "school"], negative=["
```

```
[('lisa', 0.4070283770561218)]
```

```
1 print(w2v_model.wv.most_similar(positive=["teacher", "bart"], negative=[
```

```
[('lisa', 0.41088035702705383)]
```

```
1 print(w2v_model.wv.doesnt_match(["homer", "marge", "bart", "pizza"]))
```

```
pizza
```

```
1 Start coding or generate with AI.
```

```
1 print(w2v_model.wv.doesnt_match(["burns", "smithers", "apu", "spaceship"
```

```
WARNING:gensim.models.keyedvectors:vectors for words {'spaceship'} are not pr
apu
```

Limitaciones de Word2Vec y de los embeddings estáticos

A pesar de su enorme utilidad, los embeddings tradicionales como Word2Vec presentan limitaciones importantes: